MANIPAL SCHOOL OF INFORMATION SCIENCES

LINUX OS LAB ASSIGNMENT

Advanced grep, Regex & Log Filtering Pipelines

Student Name: Ribhav Devprasad

Registration No: 261100690002

Course: Linux Operating System Lab

Part A – Basic grep and Filtering

Task Description	Command / Pipeline
Display all lines containing the word failed.	`grep -w "failed" "$LOG"`
Display all lines containing failed, ignoring case.	`grep -i "failed" "$LOG"`
Display all lines that contain either Failed or Denied.	`grep -E "Failed|Denied" "$LOG"`
Display all lines that do not contain the word INFO.	`grep -v "INFO" "$LOG"`
Display all lines containing the exact word root.	`grep -w "root" "$LOG"`
Display all occurrences of attack along with line numbers.	`grep -n "attack" "$LOG"`
Count the number of lines containing ERROR.	`grep -c "ERROR" "$LOG"`
Display only filenames containing malware.	`grep -l "malware" *.log`
Display the first five lines containing Suspicious.	`grep "Suspicious" "$LOG" | head -n 5`
Display the last five failed-login entries.	`grep -i "failed login" "$LOG" | tail -n 5`

Part B – Regular Expression Extraction

Task Description	Command / Pipeline
Extract all IPv4 addresses from the log.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG"`
Extract all IP addresses belonging to 10.x.x.x network.	`grep -Eo '\b10\.([0-9]{1,3}\.){2}[0-9]{1,3}\b' "$LOG"`
Extract all addresses belonging to 192.168.x.x network.	`grep -Eo '\b192\.168\.[0-9]{1,3}\.[0-9]{1,3}\b' "$LOG"`
Extract all email addresses.	`grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG"`
Extract email addresses ending in .com.	`grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.com\b' "$LOG"`
Extract email addresses with digits in username.	`grep -Eo '[a-zA-Z0-9._%+-]*[0-9][a-zA-Z0-9._%+-]*@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG"`
Extract all URLs beginning with http:// or https://.	`grep -Eo 'https?://[^ ]+' "$LOG"`
Extract only HTTPS URLs.	`grep -Eo 'https://[^ ]+' "$LOG"`
Extract URLs ending with .php.	`grep -Eo 'https?://[^ ]+\.php\b' "$LOG"`
Extract all CVE identifiers (CVE-YYYY-NNNN).	`grep -Eo 'CVE-[0-9]{4}-[0-9]{4,7}' "$LOG"`
Extract only CVEs reported for 2025.	`grep -Eo 'CVE-2025-[0-9]{4,7}' "$LOG"`
Extract all 32-character MD5 hashes.	`grep -Eo '\b[a-fA-F0-9]{32}\b' "$LOG"`
Extract all 40-character SHA-1 hashes.	`grep -Eo '\b[a-fA-F0-9]{40}\b' "$LOG"`
Extract all 64-character SHA-256 hashes.	`grep -Eo '\b[a-fA-F0-9]{64}\b' "$LOG"`

Extract timestamps (YYYY-MM-DD HH:MM:SS).	`grep -Eo '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}' "$LOG"`

Part C – Anchors and Regex Patterns

Task Description	Command / Pipeline
Display all lines beginning with a timestamp.	`grep -E '^[0-9]{4}-[0-9]{2}-[0-9]{2}' "$LOG"`
Display all lines beginning with Suspicious.	`grep '^Suspicious' "$LOG"`
Display lines ending with a numeric value.	`grep -E '[0-9]+$' "$LOG"`
Display lines beginning with ERROR or WARNING.	`grep -E '^(ERROR|WARNING)' "$LOG"`
Display lines whose first character is a digit.	`grep '^[0-9]' "$LOG"`
Find lines ending in .exe.	`grep '\.exe$' "$LOG"`
Find lines with files ending in .exe, .sh, or .pdf.	`grep -E '\b[a-zA-Z0-9_-]+\.(exe|sh|pdf)\b' "$LOG"`
Identify exact word root, excluding rooted.	`grep -w "root" "$LOG"`
Display blank lines in the file.	`grep '^$' "$LOG"`
Display all nonblank lines.	`grep -v '^$' "$LOG"`

Part D – Linux Filters and Pipelines

Task Description	Command / Pipeline
Extract all IPv4 addresses, sort, and deduplicate.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -u`
Count unique IPv4 addresses.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -u | wc -l`
Extract all IP addresses and display in sorted order.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -V`
Determine IP addresses appearing more than once.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq -c | awk '$1 > 1 {print $2}'`
Determine most frequently occurring IP address.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq -c | sort -nr | head -n 1`
Extract usernames from auth entries (unique).	`grep -i "user" "$LOG" | sed -E 's/.*user[=]([a-zA-Z0-9_-]+).*/\1/' | sort -u`
Count auth attempts per username.	`grep -i "user" "$LOG" | sed -E 's/.*user[=]([a-zA-Z0-9_-]+).*/\1/' | sort | uniq -c`
Extract HTTP status codes and frequency.	`grep -Eo 'HTTP/1\.[01]" [0-9]{3}' "$LOG" | awk '{print $2}' | sort | uniq -c`
Display top 5 most frequent IPs.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq -c | sort -nr | head -n 5`
Display top 3 most frequent usernames.	`grep -i "user" "$LOG" | sed -E 's/.*user[=]([a-zA-Z0-9_-]+).*/\1/' | sort | uniq -c | sort -nr | head -n 3`

Extract destination ports in numerical order.	`grep -Eo 'port [0-9]+' "$LOG" | awk '{print $2}' | sort -n`
Remove duplicate port numbers.	`grep -Eo 'port [0-9]+' "$LOG" | awk '{print $2}' | sort -nu`
Count unique destination ports.	`grep -Eo 'port [0-9]+' "$LOG" | awk '{print $2}' | sort -u | wc -l`
Extract file extensions and frequency.	`grep -Eo '\.[a-zA-Z0-9]{2,4}\b' "$LOG" | sort | uniq -c | sort -nr`
Find top 5 most frequent domains.	`grep -Eo '([a-zA-Z0-9-]+\.)+[a-zA-Z]{2,}' "$LOG" | sort | uniq -c | sort -nr | head -n 5`

Part E – Combining grep Conditions

Task Description	Command / Pipeline
Display lines containing IP address and Failed.	`grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | grep "Failed"`
Display lines containing both root and Failed.	`grep "root" "$LOG" | grep "Failed"`
Display lines containing download or upload.	`grep -E "download|upload" "$LOG"`
Display suspicious entries with .exe or .sh.	`grep -i "suspicious" "$LOG" | grep -E '\.(exe|sh)\b'`
Display lines with CVE identifier and IP.	`grep -E 'CVE-[0-9]{4}-[0-9]{4,7}' "$LOG" | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}'`
Display lines with filename and hash value.	`grep -E '\b[a-zA-Z0-9_-]+\.[a-zA-Z0-9]{2,4}\b' "$LOG" | grep -E '\b[a-fA-F0-9]{32,64}\b'`
Display IP address followed by port.	`grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}.*port [0-9]+' "$LOG"`
Display HTTP requests with URL and 404 status.	`grep -E 'https?://[^ ]+' "$LOG" | grep ' 404 '`
Display auth failures excluding 127.0.0.1.	`grep -i "failed" "$LOG" | grep -v '127\.0\.0\.1'`
Display attack entries excluding INFO.	`grep -i "attack" "$LOG" | grep -v "INFO"`

Part F – Negative Matching

Task Description	Command / Pipeline
Extract IPv4 addresses except 127.x.x.x.	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | grep -v '^127\.'`
Display log entries except successful logins.	`grep -iv "success" "$LOG"`
Exclude INFO, DEBUG, or SUCCESS entries.	`grep -Ev "INFO|DEBUG|SUCCESS" "$LOG"`
Find HTTP requests with status code not 200.	`grep 'HTTP/' "$LOG" | grep -v ' 200 '`
Display suspicious traffic excluding 192.168.x.x.	`grep -i "suspicious" "$LOG" | grep -v '192\.168\.'`
Extract emails excluding company domain.	`grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG" \| grep -v '@company\.com$'`

Part G – Redirection & Combining Filters

Task Description	Command / Pipeline
Extract IPv4 addresses to ips.txt	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" > ips.txt`
Extract email addresses to emails.txt	`grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG" > emails.txt`
Extract URLs to urls.txt	`grep -Eo 'https?://[^ ]+' "$LOG" > urls.txt`
Extract hashes to hashes.txt	`grep -Eo '\b[a-fA-F0-9]{32,64}\b' "$LOG" > hashes.txt`
Extract failed logins to failed_login.txt	`grep -i "failed login" "$LOG" > failed_login.txt`
Append suspicious entries to incident_report.txt	`grep -i "suspicious" "$LOG" >> incident_report.txt`
Store unique IP addresses in unique_ips.txt	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -u > unique_ips.txt`
Store IP frequency in ip_frequency.txt	`grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq -c | sort -nr > ip_frequency.txt`
Redirect output to results.txt and errors to errors.txt	`grep "ERROR" "$LOG" 1> results.txt 2> errors.txt`
Extract lowercase unique domains to domains.txt	`grep "Domain accessed" "$LOG" | sed -E 's/.*Domain accessed:? //I' | tr '[:upper:]' '[:lower:]' | sort -u > domains.txt`
Save unique failed-login IPs to failed_login_ips.txt	`grep -i "failed login" "$LOG" | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort -u > failed_login_ips.txt`
Extract failed-login usernames to failed_users.txt	`grep -i "failed login" "$LOG" | sed -E 's/.*user[=]([a-zA-Z0-9_-]+).*/\1/' | sort -u > failed_users.txt`
Extract uppercase MD5 hashes to md5_hashes.txt	`grep -Eo '\b[a-fA-F0-9]{32}\b' "$LOG" | tr '[:lower:]' '[:upper:]' > md5_hashes.txt`
Strip square brackets to clean_log.txt	`tr -d '[]' < "$LOG" > clean_log.txt`
Convert log file to lowercase normalized_log.txt	`tr '[:upper:]' '[:lower:]' < "$LOG" > normalized_log.txt`
Find failed-login usernames in uppercase	`grep -i "failed login" "$LOG" | sed -E 's/.*user[=]([a-zA-Z0-9_-]+).*/\1/' | tr '[:lower:]' '[:upper:]' | sort -u`
Extract User-Agent field in lowercase	`grep "User-Agent:" "$LOG" | sed -E 's/.*User-Agent: ?//I' | tr '[:upper:]' '[:lower:]'`
Extract .exe filenames in uppercase	`grep -i "\.exe" "$LOG" | grep -Eo '\b[a-zA-Z0-9_-]+\.exe\b' | tr '[:lower:]' '[:upper:]'`